手机性能采样机制与指标解释

AppPerformanceMonitor 采集当前 App 进程的性能近似状态，约每秒生成一份快照。CPU、PSS、堆内存与 FPS 使用不同测量口径，不能互相代替。

适用分支 p/caijiamin/feat_debug_2th
基线提交 9010928ca51c6e766f75414c49c748338e0b7ecb
资料日期 2026 年 11 月 3 日

采样何时启动

采集器监听 Application.ActivityLifecycleCallbacks，以 startedActivityCount 记录 started 的 Activity 数。计数由零变一时启动；全部停止后取消采样与 Choreographer 帧回调。

采样通过主线程 Handler 以 1000 毫秒延迟循环。实际间隔可能因主线程调度变长，计算使用相邻采样的 elapsedRealtime 差值，而不是直接假设恰好一秒。

CPU 使用率

分子是 Process.getElapsedCpuTime 的增量，分母是经过时间再乘可用 CPU 核数，最终乘 100 并限制在 0 到 100。该值是按核心数归一化的进程 CPU 指标。

它不是整机总 CPU 占用，也不能与没有除核心数的监控工具直接比较。排查时同时记录设备核心数、采样时长和构建类型，避免把口径差异当作采样错误。

内存口径

PSS 从 ActivityManager.getProcessMemoryInfo 获取当前 PID 的 totalPss，除以 1024 显示为 MB。Java Heap 为 Runtime.totalMemory 减 freeMemory，再除以 1024 的平方；Native Heap 使用 Debug.getNativeHeapAllocatedSize。

这些字段不满足简单相加等于 PSS 的关系。PSS 读取失败时当前实现以零参与构造快照，这与明确的空值状态不同；解读为零内存使用之前应核实采集是否成功。

帧率和温度

FPS 是采样期间 Choreographer 回调次数乘 1000 再除以实际毫秒数。它表示应用帧回调频率，不是相机传感器帧率、编码器输出帧率或直播接收端帧率。

电池温度通过 ACTION_BATTERY_CHANGED 最近广播读取 EXTRA_TEMPERATURE，除以 10 得到摄氏度。无广播或缺失温度返回 null；不能将它称作 CPU 温度。

采集和历史数据边界

latestSnapshot 只保存最近一次结果，监听者注册后可立即收到它。该类没有长时间采样持久化、CSV 累积或历史区间查询。二期性能导出需要另建带时间戳的记录能力。

release 在主线程解除生命周期注册，停止采样、清空监听和最新值。验证应覆盖前后台、多个 Activity 重叠 started、屏幕旋转及释放后无回调。

源码依据与版本回查

以下路径相对于 OBSBOT_Live 仓库。Impl 表示 business/debug/debug_impl/src/main/java/com/obsbot/live/business/debug，API 表示对应 debug_api 源码根。摘要为文件 SHA256 前 16 位。

Impl/AppPerformanceMonitor.kt
摘要 2462ff4554747d8a；符号 PerformanceSnapshot、AppPerformanceMonitor、doFrame、run、register、addListener

Impl/data/source/PhonePerformanceSource.kt
摘要 15b1991571c54a4e；符号 PhonePerformanceSource、register、update